WYDZIAŁ TELEKOMUNIKACJI, INFORMATYKI I

ELEKTROTECHNIKI

Metody ochrony cyberprzestrzeni - projekt

SQL Injection



Krzysztof Demski

nr alb. 108715

Informatyka stosowana II stopień, semestr III.

1. Wstęp

SQL Injection - jest to dosyć częsty atak najczęściej przeprowadzany na aplikacjach webowych, polegający na tym, że atakujący wstrzykuje do aplikacji nieautoryzowany fragment kodu z zapytania SQL.

Wstrzyknięcie jest możliwe w przypadku braku odpowiedniego mechanizmu walidującego parametry przekazywane przez użytkownika.

Podstawowym scenariuszem ataku SQL injection jest umieszczenie złośliwego kodu SQL w formularzu na stronie internetowej lub innej aplikacji, gdzie dane wejściowe są niewystarczająco zabezpieczone. Gdy atakujący przekazuje dane do serwera, a aplikacja nie sprawdza ich dostatecznie, kod SQL wstrzykiwany przez atakującego może być wykonywany przez bazę danych.

Przykładowe zapytanie SQL w języki PHP odczytujące dane z formularza:

“SELECT * FROM USERS WHERE user=’$user’ AND pass=’$pass’”);

Gdzie zmienna $user i $pass to dane przesłane przez użytkownika wypełniającego formularz np: logowania.
Jeżeli mamy do czynienia z atakiem hakerskim, gdy atakujący wpiszę w polach login i hasło ‘or’1’=’1 to zapytanie SQL przyjmie następującą sekwencję:

“SELECT * FROM USERS WHERE user=’ ‘or’1’=’1 ’ AND pass=’ ‘or’1’=’1 ’”);

Odpowiednimi kolorami zaznaczono interpretacje znaków apostrof. Poprzez podanie ‘or’1’=’1 w loginie i haśle zapytanie SQL całkowicie zmieniło sens. W warunku WHERE sprawdzane jest czy pole user jest puste (fałsz) lub 1 = 1 (prawda) i pole pass jest puste (fałsz) lub 1=1 (prawda). Dzięki operatorowi OR całe zapytanie zwraca wartość 1 gdyż oba warunki klauzuli WHERE są spełnione.

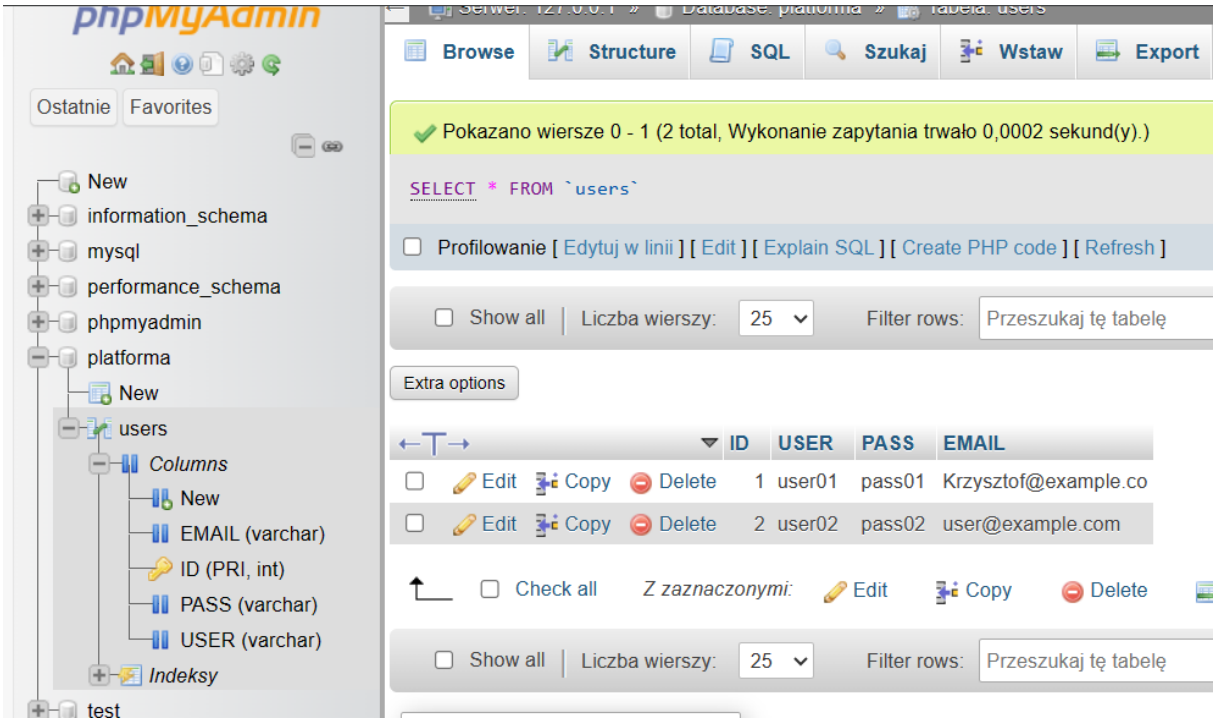
Taka sytuacja może skutkować sytuacją, w której atakujący uzyskał nieautoryzowany dostęp do danych (ominięcie mechanizmu uwierzytelniania). Atakujący może być w stanie utworzyć, zmodyfikować lub nawet usunąć dane w bazie danych posługując się kodem SQL.

2. Przygotowanie danych do realizacji ataku

Żeby przeprowadzić próbę ataku hakerskiego został utworzony lokalny serwer webowy przy pomocy oprogramowania XAMPP. Sam atak został przeprowadzony na serwerze localhost.

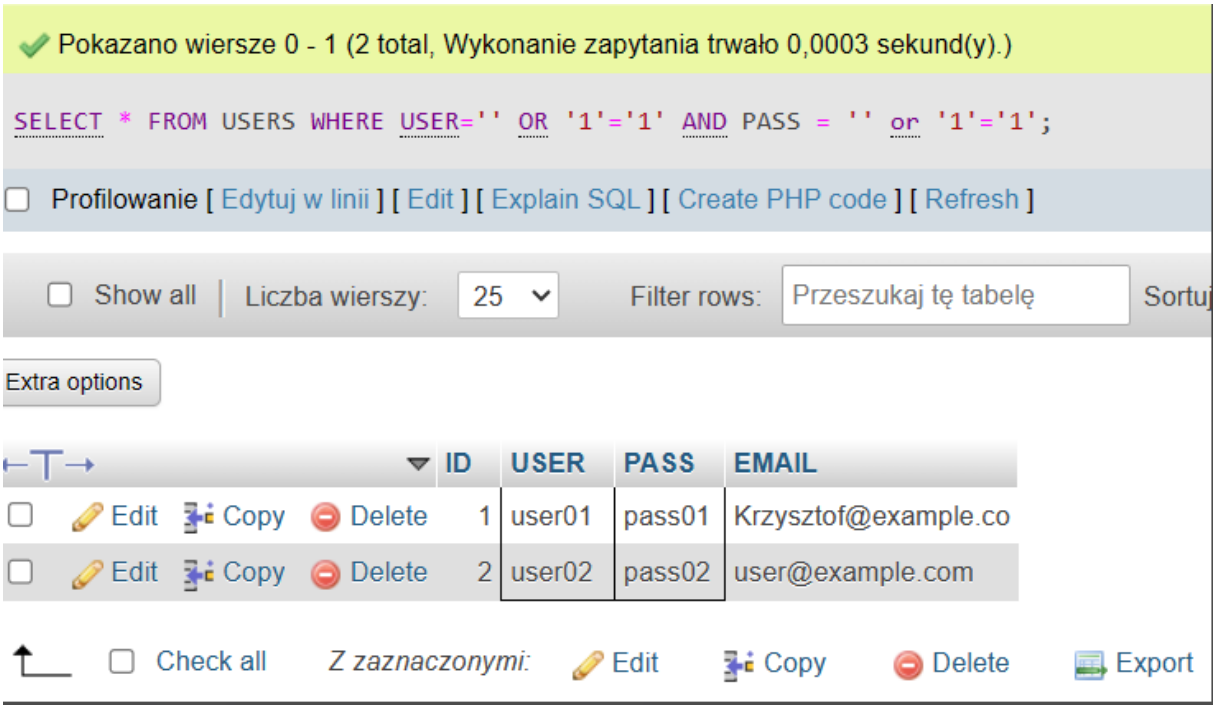
W phpMyAdmin utworzono bazę danych o nazwie platforma i dodano do niej jedną tabelę – users. Tabela ta posiadała cztery kolumny (ID, USER, PASS, EMAIL).

W normalnych warunkach pole hasło byłoby zaszyfrowane metodą MD5, jednak dla przeprowadzenia ataku SQL Injection nie ma to znaczenia.



Następnie do tabeli zostały dodane dwa rekordy. Dwóch użytkowników, gdzie pole USER oznacza login użytkownika a pole PASS jego hasło.

W kolejnym kroku przeprowadzono test (przykład omówiony w punkcie pierwszym) i do phpMyAdmin został wprowadzone zapytanie SQL:



Jak widać interpreter w phpMyAdmin również wyświetlił że zapytanie SQL jest prawdziwe.

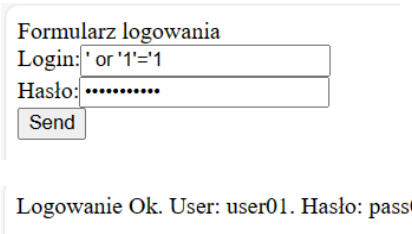
3. Przeprowadzenie ataku

Żeby przeprowadzić atak zostały utworzony formularz logowania, który po prawidłowej walidacji przeprowadzi użytkownika do kolejnej strony.
Poniżej został przedstawiony kod źródłowy formularza logowania podatnego na atak SQL Injection napisany w języku PHP

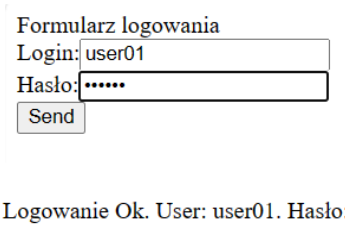
```
$user=$_POST['user']; // login z formularza
$pass=$_POST['pass']; // hasło z formularza
$dbhost="localhost";
$dbuser="root";
$dbpassword="";
$dbname="platforma";
$polaczenie = new mysqli($dbhost, $dbuser, $dbpassword, $dbname);
if(!$polaczenie) { echo"Error: ". mysqli_connect_errno()." ".mysqli_connect_error();
} // obsługa błędu połączenia z BD

   $rezultat=$polaczenie->query( SELECT * FROM users WHERE user='$user'
AND pass='$pass'"); // Zapytanie SQL przekazujące dane do bazydanych
$rekord = @mysqli_fetch_array($rezultat); // pobieranie wiersza z BD, struktura
zmiennej jak w BD
if(!$rekord) //Jeśli brak, to nie ma użytkownika o podanym loginie
{
$polaczenie->close();; // zamknięcie połączenia z BD
echo "Blad nazwy użytkownika lub hasla";
}
else
{
echo "Logowanie Ok. User: {$rekord['USER']}. Hasło: {$rekord['PASS']}"; // Jeśli
$rekord istnieje }
```

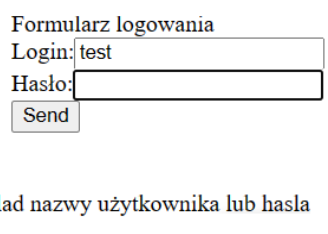
Wyniki powyższego skryptu przedstawiono na poniższym zrzucie ekranu. Zgodnie z założeniami wpisanie sekwencji ‘or’1’=’1 do loginu i hasła pozwoliło na zalogowanie się w sposób nieautoryzowany do środka aplikacji.



Logowanie Ok. User: user01. Hasło: pass01



Logowanie Ok. User: user01. Hasło: pass01



Blad nazwy użytkownika lub hasla

4. Zabezpieczenie skryptu przed atakiem SQL Injection

W celu zabezpieczenia powyższego skryptu została przebudowana część składni PHP a walidacja odbywa się dwuetapowo tzn najpierw weryfikowany jest login a dopiero później następuje walidacja hasła. Poniżej przedstawiono zmodyfikowany skrypt:

```
if(!$link) { echo"Błąd: ". mysqli_connect_errno()." ".mysqli_connect_error(); } // obsługa błędu połączenia z BD

mysqli_query($link, "SET NAMES 'utf8'"); // ustawienie polskich znaków

$result = mysqli_query($link, "SELECT * FROM users WHERE USER='$user'"); // pobranie z BD wiersza, w którym login=login z formularza [pierwszy etap walidacji]

$rekord = mysqli_fetch_array($result); // wiersza z BD, struktura zmiennej jak w BD

if(!$rekord) //Jeśli brak, to nie ma użytkownika o podanym loginie

{ mysqli_close($link); // zamknięcie połączenia z BD

echo "Brak użytkownika o takim loginie !"; // UWAGA nie wyświetlamy takich podpowiedzi dla hakerów

}

else

{ // Jeśli $rekord istnieje

if($rekord['PASS']==$pass) // czy hasło zgadza się z BD [2 etap walidacji]

{ echo "Logowanie Ok. User: {$rekord['USER']}. Hasło: {$rekord['PASS']}";

}

else

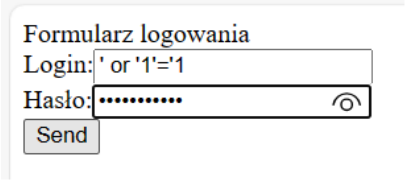
{ mysqli_close($link);

echo "Błąd w haśle !";

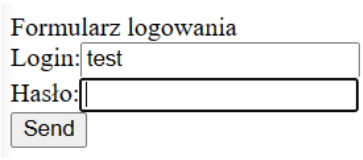
}

}
```

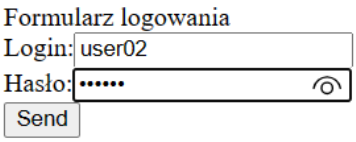
Poniżej przedstawiono zrzut ekranu wynikami powyższego skryptu. Jak można zauważyć dla powyższego skryptu wpisanie ‘ or ‘1’=’1 do loginu i hasła już nie wystarczyło. Formularz został zabezpieczony przed taką formą ataku.



Błąd w haśle !



Brak użytkownika o takim loginie !



Logowanie Ok. User: user02. Hasło: pass02

5. Wnioski i spostrzeżenia

W niniejszej pracy przedstawiono dwa formularze, działające dokładnie tak samo. Przekierowują one użytkownika na inną stronę w momencie gdy ten prawidłowo się uwierzytelni. Jednak inne metody walidacji sprawiają że jeden formularz jest podatny na atak SQL Injection, drugi zaś jest odporny.

Porównując skrypty walidacyjne można dojść do następujących wniosków:

Należy unikać bezpośredniego łączenia danych wprowadzonych przez użytkownika z zapytaniem SQL. Skrypty mogą być pisane w oparciu o parametry, które są zabezpieczone przed atakami. Innym sposobem jest uwierzytelnienie tylko loginu, w przypadku pomyślnej weryfikacji w oddzielnym kroku weryfikowane jest hasło.

Kolejnym pomysłem jest wprowadzenie ograniczenia na wpisywanie potencjalnie niebezpiecznych znaków np.: ‘ ; „ ` , . Za pomocą symboli ‘ ‘ atak typu SQL Injection został przeprowadzony pomyślnie.